Intro to Data Science - HW 6

IST 687

Kent Roller

Due Date: 2/23/24

Submission Date: 2/22/24

```
# Enter your name here: Kent Roller
```

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this homework by myself, with help from the book and the professor.
```

Last assignment we explored **data visualization** in R using the **ggplot2** package. This homework continues to use ggplot, but this time, with maps. In addition, we will merge datasets using the built-in **merge( )** function, which provides a similar capability to a **JOIN in SQL** (don't worry if you do not know SQL). Many analytical strategies require joining data from different sources based on a **"key"** – a field that two datasets have in common.

Step 1: Load the population data

A. Read the following JSON file, https://intro-datascience.s3.us-east-2.amazonaws.com/cities.json (https://intro-datascience.s3.us-east-2.amazonaws.com/cities.json) and store it in a variable called **pop**.

Examine the resulting pop dataframe and add comments explaining what each column contains.

```
# Loading packages expecting to use
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✓ dplyr     1.1.3     ✓ readr     2.1.4
## ✓ forcats   1.0.0     ✓ stringr   1.5.0
## ✓ ggplot2   3.4.4     ✓ tibble    3.2.1
## ✓ lubridate 1.9.3     ✓ tidyr     1.3.0
## ✓ purrr     1.0.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(jsonlite)
```

```
## Warning: package 'jsonlite' was built under R version 4.3.2
```

```
##
## Attaching package: 'jsonlite'
##
## The following object is masked from 'package:purrr':
##
##     flatten
```

```
# Importing the file from the webpage
pop<-fromJSON("https://intro-datascience.s3.us-east-2.amazonaws.com/cities.json", flatten=TRUE)
```

```
# Looking at the variables in the file
head(pop)
```

```
##          city growth_from_2000_to_2013 latitude  longitude population rank
## 1    New York                     4.8% 40.71278  -74.00594    8405837    1
## 2  Los Angeles                     4.8% 34.05223 -118.24368    3884307    2
## 3     Chicago                    -6.1% 41.87811  -87.62980    2718782    3
## 4     Houston                    11.0% 29.76043  -95.36980    2195914    4
## 5 Philadelphia                     2.6% 39.95258  -75.16522    1553165    5
## 6     Phoenix                    14.0% 33.44838 -112.07404    1513367    6
##          state
## 1     New York
## 2   California
## 3     Illinois
## 4        Texas
## 5 Pennsylvania
## 6      Arizona
```

```
# Looking at the output we have the following variables in the dataframe pop
# City: List the name of the city in a character string
# Growth_from_2000_to_2013: List the percent change in growth of the population in the city as measured from 2000 to 2013.
# Latitude: List the latitude of the city
# Longitude: List the longitude of the city
# Population: List the population as a numeric value for each city however it is stored as a character string which means it
will need to be converted before any type of insight can be gained via analysis.
# Rank: A ranked list of the populations where the highest population is ranked number 1 and the smallest population is rank
1000.
# State: The name of the state that the city is located in
```

B. Calculate the **average population** in the dataframe. Why is using mean() directly not working? Find a way to correct the data type of this variable so you can calculate the average (and then calculate the average)

Hint: use **str(pop)** or **glimpse(pop)** to help understand the dataframe

```
# As noted previously any type of numeric manipulation will not work as the population variable is stored by default as a ch
aracter string, and as such is recognized as a text string, not a list of numbers. Need to change with pipes and as.numeric
or as.dbl to remedy.
pop<- pop %>% mutate(population=as.numeric(population))
mean(pop$population)
```

```
## [1] 131132.4
```

```
# The mean population obtained from the output is 131,132.4 persons.
```

C. What is the population of the smallest city in the dataframe? Which state is it in?

```
#

pop.small.test<- pop %>% filter(rank==1000)
pop.small.test
```

```
##          city growth_from_2000_to_2013 latitude longitude population rank
## 1 Panama City                     0.1% 30.15881 -85.66021      36877 1000
##     state
## 1 Florida
```

```
# Probably a long way to do this, messy, but:
pop.small<- pop %>% filter(population == min(population, na.rm=TRUE))
pop.small
```

```
##          city growth_from_2000_to_2013 latitude longitude population rank
## 1 Panama City                     0.1% 30.15881 -85.66021      36877 1000
##     state
## 1 Florida
```

```
# Could have put in order and then found it that way as well.

pop.smallest<-pop[1000,]
pop.smallest
```

```
##             city growth_from_2000_to_2013 latitude longitude population rank
## 1000 Panama City                     0.1% 30.15881 -85.66021      36877 1000
##        state
## 1000 Florida
```

```
# Seems they may already be in order by rank

#The city with the smallest population is Panama City in the state of Florida with a population of 36877.
```

Step 2: Merge the population data with the state name data

D. Read in the state name .csv file from the URL below into a dataframe named **abbr** (for “abbreviation”) – make sure to use the read_csv() function from the tidyverse package:
https://intro-datascience.s3.us-east-2.amazonaws.com/statesInfo.csv (https://intro-datascience.s3.us-east-2.amazonaws.com/statesInfo.csv)

```
abbr<-read_csv("https://intro-datascience.s3.us-east-2.amazonaws.com/statesInfo.csv")
```

```
## Rows: 51 Columns: 2
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## chr (2): State, Abbreviation
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

E. To successfully merge the dataframe **pop** with the **abbr** dataframe, we need to identify a **column they have in common** which will serve as the **“key”** to merge on. One column both dataframes have is the **state column**. The only problem is the slight column name discrepancy – in **pop**, the column is called **“state”** and in **abbr** – **“State.”** These names need to be reconciled for the merge() function to work. Find a way to rename **abbr’s “State”** to **match** the **state column in pop**.

```
abbr<- abbr %>% rename(state=State)
```

F. Merge the two dataframes (using the **‘state’ column** from both dataframes), storing the resulting dataframe in **dfNew**.

```
df.new<-inner_join(pop, abbr, by="state")
```

G. Review the structure of **dfNew** and explain the columns (aka attributes) in that dataframe.

```
head(df.new)
```

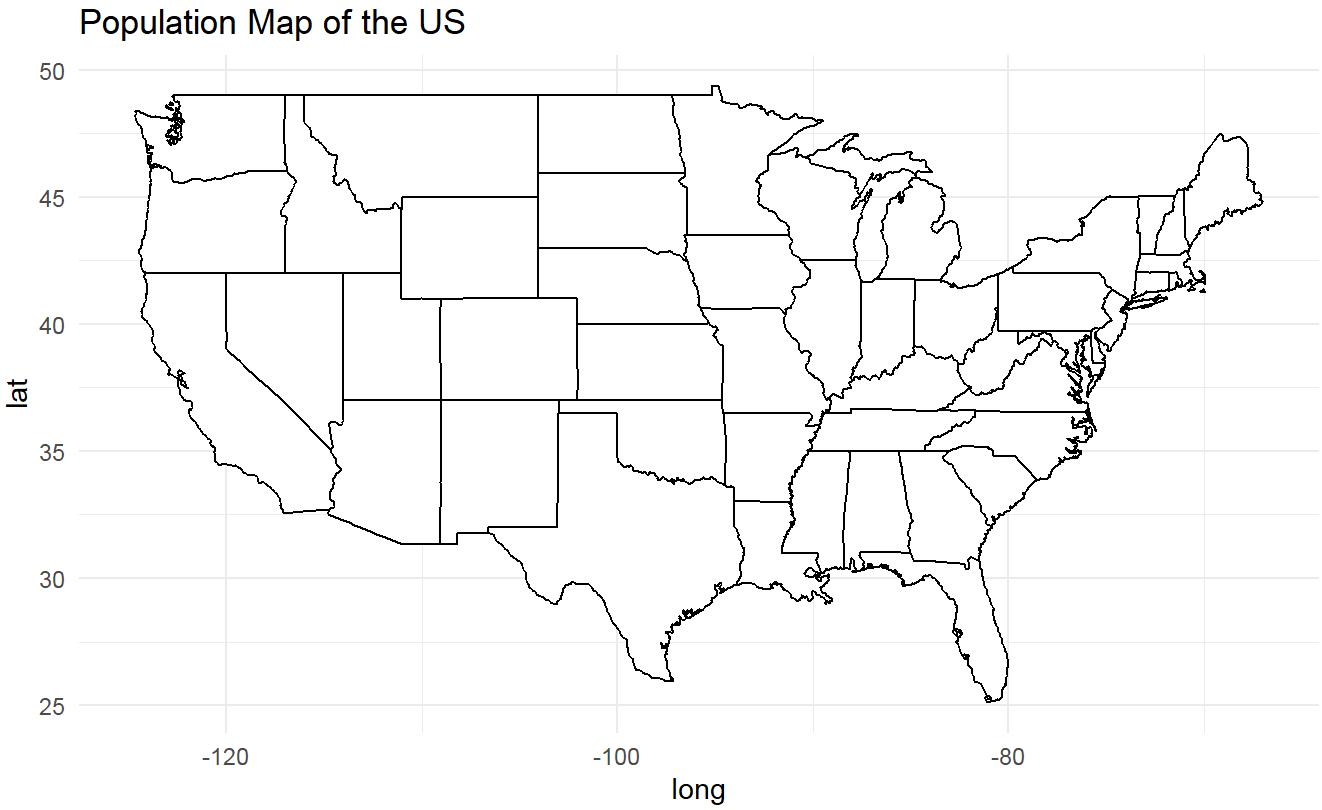
```
##           city growth_from_2000_to_2013 latitude  longitude population rank
## 1     New York                     4.8% 40.71278  -74.00594    8405837    1
## 2  Los Angeles                     4.8% 34.05223 -118.24368    3884307    2
## 3      Chicago                    -6.1% 41.87811  -87.62980    2718782    3
## 4      Houston                    11.0% 29.76043  -95.36980    2195914    4
## 5 Philadelphia                     2.6% 39.95258  -75.16522    1553165    5
## 6      Phoenix                    14.0% 33.44838 -112.07404    1513367    6
##           state Abbreviation
## 1     New York           NY
## 2   California           CA
## 3     Illinois           IL
## 4        Texas           TX
## 5 Pennsylvania           PA
## 6      Arizona           AZ
```

```
# The variables in this new dataframe are all identical to the previous dataframe pop, the only change is with the new varia
ble called Abbreviation, which is a charater string of the two letter abbreviations for each state.
```

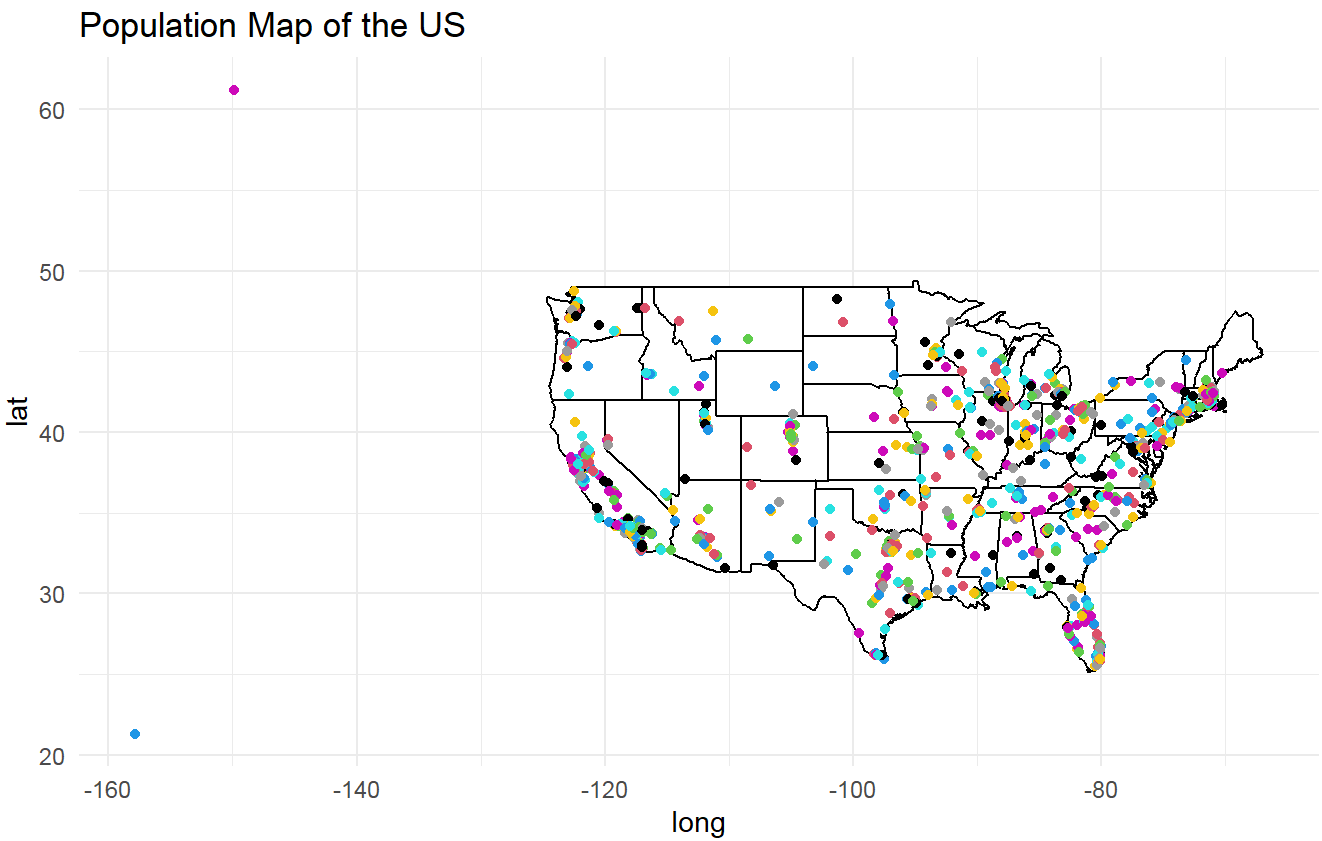
Step 3: Visualize the data

H. Plot points (on top of a map of the US) for **each city**. Have the **color** represent the **population**.

```
# Make a map with ggplot
us.map<- map_data("state")
ggplot() + geom_polygon(data=us.map, aes(x=long, y=lat, group=group), fill="white", color="black")+ theme_minimal() +
  labs(title="Population Map of the US") + coord_fixed(1.3)
```




```
# Now to layer additional information over map
ggplot() +
  geom_polygon(data = us.map, aes(x = long, y = lat, group = group), fill = "white", color = "black") +
  geom_point(data = pop, aes(x = longitude, y = latitude), color = pop$population) +
  theme_minimal() +
  labs(title = "Population Map of the US") +
  coord_fixed(1.3)
```



I. Add a block comment that criticizes the resulting map. It's not very good.

```
# The map has a poor zoom level, and also does not include outlines for the states of AK or HA. There is also no legend, so there is no indication of what values the dots represent in terms of color (however it should be noted that this is most likely an issue to do with the current zoom level). And of course, all of these things being true, this map is not very informative, as in, it tells the viewer nothing aside from being able to match dots of the same color, but with no context that doesn't tell the viewer much.
```

Step 4: Group by State

J. Use group_by and summarise to make a dataframe of state-by-state population. Store the result in **dfSimple**.

```
df.simple<- pop %>% group_by(state) %>% summarise(state_pop =sum(population, na.rm=TRUE))
```

K. Name the most and least populous states in **dfSimple** and show the code you used to determine them.

```
pop.small2<- df.simple %>% filter(state_pop == min(state_pop, na.rm=TRUE))
pop.small2
```

```
## # A tibble: 1 × 2
##   state   state_pop
##   <chr>       <dbl>
## 1 Vermont     42284
```

```
pop.big<-df.simple %>% filter(state_pop == max(state_pop, na.rm=TRUE))
pop.small2
```

```
## # A tibble: 1 × 2
##   state   state_pop
##   <chr>       <dbl>
## 1 Vermont     42284
```

```
pop.big
```

```
## # A tibble: 1 × 2
##   state      state_pop
##   <chr>          <dbl>
## 1 California  27910620
```

```
# The state with the smallest population is Vermont at 42,284 and the state with the largest population is California at 27,
910,620.
```

Step 5: Create a map of the U.S., with the color of the state

representing the state population

L. Make sure to expand the limits correctly and that you have used **coord_map** appropriately.

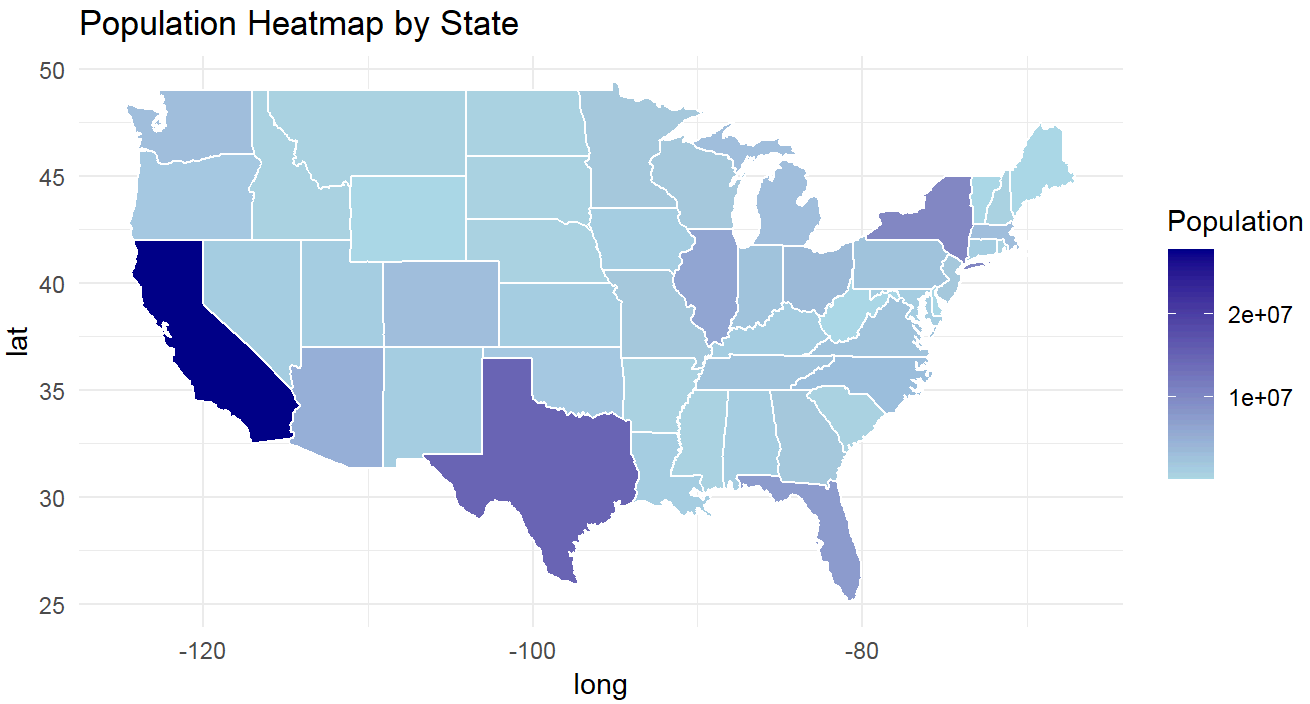
```
states.map<- map_data("state")

df.simple2<-df.simple
```

```
df.simple2$state<-tolower(df.simple2$state)
```

```
df.simple2<- df.simple2 %>% rename(region=state)
```

```
merged.map<- merge(states.map, df.simple2, by.x="region", by.y="region")
```

```
ggplot(merged.map) +
  geom_polygon(aes(x = long, y = lat, group = group, fill = state_pop), color = "white") +
  scale_fill_gradient(low = "lightblue", high = "darkblue", name = "Population") +
  labs(title = "Population Heatmap by State") +
  coord_fixed(1.3) +
  theme_minimal()
```


Population Heatmap by State
50
45
40
35
30
25
lat
-120
-100
-80
long
Population
2e+07
1e+07